Twitter & Reddit Sentiment Analysis Using PySpark

An End-to-End Pipeline for Social Media Text Classification

Bob Philip

African Institute for Mathematical Sciences (AIMS-Rwanda)

November 28, 2025

Supervised by: Dr.Lema Logamou

Motivation: Why Analyze Social Media Sentiment?

The Challenge:

- Millions of public opinions are shared daily across platforms like **Twitter** and **Reddit**.
- Manually monitoring and ingesting the information is a huddle.

The Value:

- **Market Research:** Track public opinion on products or services.
- **Political Science:** Gauge public support for policies or figures.
- **Crisis Management:** Identify and react to negative trends quickly.

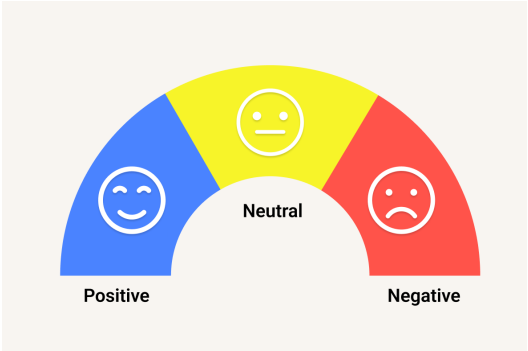


Figure 1: Target Classification

Project Scope & Core Technologies

- **Goal:** Build a robust, end-to-end pipeline to classify raw text into **Positive (1)**, **Neutral (0)**, and **Negative (-1)** sentiment categories.
- **Data Sources:** Combined text comments and posts from **Twitter** and **Reddit**. (Total $\approx 230,000$ records)
- **Scalability: PySpark**
  - Utilized **PySpark** for efficient, distributed processing of large text data (NLP, Feature Engineering).
- **Methodology:** Classical Machine Learning (ML) classifiers trained on **TF-IDF** features.

Initial Data Snapshot: Label Distribution

Raw Data Observations:

- Loaded 230,436 total records.
- The dataset included missing values and null labels that required handling.
- The distribution of the raw sentiment labels $(+1, 0, -1)$ shows the initial balance (or imbalance) of the dataset.

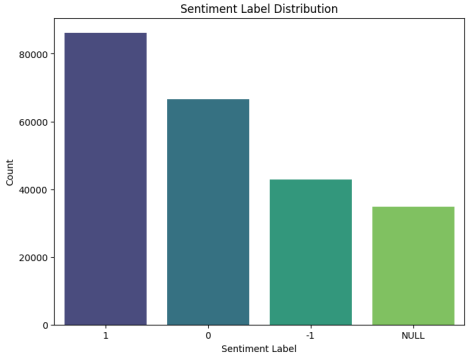


Figure 2: Sentiment Label Distribution

Exploratory Data Insight: Message Length

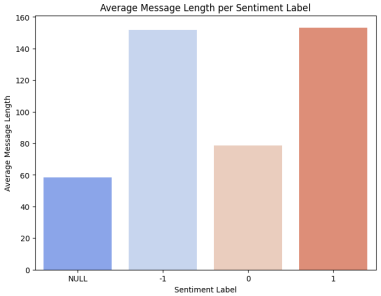


Figure 3: Message Lengths

Key Finding:

- The average length of messages varies across sentiment categories.
- **Positive** and **Negative** messages tend to be slightly longer than **Neutral** messages, suggesting more context or complex opinions in polarized posts.
- This confirms that text characteristics (length, complexity) are relevant features.

PySpark Data Cleaning Pipeline

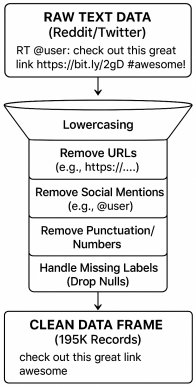
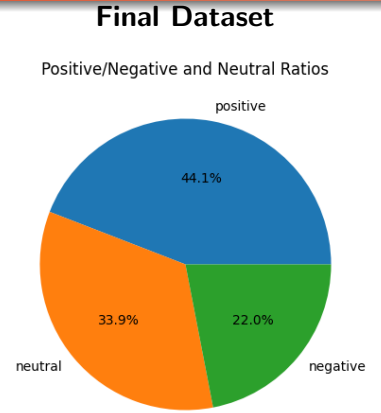


Figure 4: Cleaning Pipeline

- **Lowercasing** all text.
- Removal of:
  - URLs (`http(s)://...`)
  - Social media mentions (`@username`)
  - HTML tags and characters (e.g., `&`)
  - Numbers and punctuation.
- **Result:** Dropped 34,755 null/invalid labels, retaining **195,323 clean** rows for model training.

Final Data Distribution for Model Training



Dataset Overview:

- The cleaned dataset of $\approx$ 195K rows was split into **85% Training** and **15% Testing**.
- The final class balance is important for robust classification.
- The model is trained to recognize the relationship between text features and these three distinct sentiment labels.

Figure 5: Final Features Distribution

Initial Text Exploration: Word Cloud

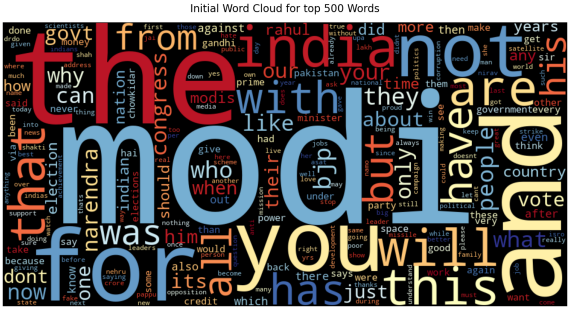


Figure 6: Initial Word Cloud

Observation on Raw Text:

- Visualizing the most frequent words from the **uncleaned** dataset.
- Reveals common, irrelevant terms (e.g., proper nouns, articles, prepositions).
- This reinforces the need for **StopWord Removal** to focus the classifier on meaningful sentiment indicators.

Feature Engineering: Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF Pipeline in PySpark ML:

1. **Tokenization:** Splitting text into individual words (tokens).
2. **StopWord Removal:** Eliminating common words (e.g., 'a', 'the', 'is').
3. **CountVectorizer:** Converts tokens into a vector of raw term frequencies (TF).
4. **IDF Transformer:** Weights the TF vector by the Inverse Document Frequency (IDF), highlighting unique, important terms.

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF
Term x within message y

$tf_{x,y}$ = frequency of word x in message y
df_x = number of messsages containing x
N = total number of messages

Figure 7: TF-IDF

Insight: Key Vocabulary for POSITIVE Messages

Figure 8: Cleaned positives Word Cloud

Contextual Findings:

- The TF-IDF features, when analyzed for the positive class, highlight terms strongly associated with approval, support, and optimism.
- This visualization helps **verify the quality** of the feature engineering process.
- The model primarily learns from these context-specific terms, not general English vocabulary.

Insight: Key Vocabulary for NEGATIVE Messages

Contextual Findings:

- Similarly, analysis of the negative class reveals a distinct vocabulary of dissatisfaction, criticism, and anger.
- Examples might include terms related to specific complaints or negative descriptors.
- **This contrast is what the classifier learns to exploit.**

Figure 9: Cleaned negative words

Model Training

PySpark ML Classifiers Tested:

- **Logistic Regression (LR):** A strong, fast linear model for sparse, high-dimensional data (like TF-IDF vectors).
- **Random Forest (RF):** An ensemble method capable of capturing complex, non-linear relationships.
- **Naive Bayes NB** A supervised machine learning algorithm that uses Bayes' theorem to predict the class of a new data point

Training Process:

- The models were trained within the PySpark ML pipeline framework.
- Evaluated on the held-out **15%** Test Set.

Model Evaluation: Performance Comparison and Selection

Metrics: We primarily used **Accuracy** (overall correctness) and the **F1 Score** (harmonic mean of precision and recall) for classification performance.

Classifier	Accuracy	F1 Score
Logistic Regression	**0.8103**	**0.8101**
Naive Bayes	0.7003	0.7032
Random Forest	0.4464	0.2758

Table 1: Performance on the Test Set

Selection: The **Logistic Regression** model significantly outperformed Random Forest and Naive Bayes, proving to be the most effective for this TF-IDF feature set.

Conclusion and Key Takeaways

- **Successful Classification:** The PySpark pipeline successfully trained a sentiment classifier, achieving $\sim$ **81**% F1 score with Logistic Regression.
- **Scalability Demonstrated:** PySpark was effective for the entire end-to-end process: loading large data, distributed cleaning, feature engineering, and model training.
- **The Importance of Prep:** Rigorous text cleaning and TF-IDF feature extraction were crucial for achieving high performance.

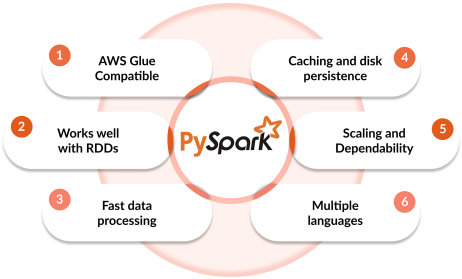


Figure 10: Features of Pyspark

Tweepy Implementation

Integrated the trained model with the live Twitter streaming API to provide **real – time** sentiment predictions.

tweepy™

Link to App

Questions

- **Optimization:** Implement **Hyperparameter Tuning** (e.g., cross-validation grid search) to further improve the Logistic Regression model.
- **Advanced Modeling:** Explore deep learning techniques like **Word2Vec** or **BERT**) for more training representation.
- **Real-Time Application:** Integrate the trained model with the live Twitter streaming for adv sentiment predictions.

Thank You! Questions?